Autoformazione sullo sviluppo front end

React Native (Expo) e React: due progetti sviluppati e rivisti

Preparato da Salvatore · Aggiornato al 4 novembre 2026

In sintesi

- Nel periodo di non allocazione su progetto ho investito il tempo nell'autoformazione sullo sviluppo front end, con due applicazioni complete e funzionanti.
- **Project Hub**: app mobile per la gestione di attività di progetto, con **React Native ed Expo** (TypeScript).
- **Customer Journey Dashboard**: applicazione web full stack (React, Node.js/Express, SQLite) per analizzare il funnel di conversione e i canali di acquisizione. Vicina al lavoro di analisi: parte dai dati e arriva a indicatori leggibili.
- Di entrambi ho rivisto il codice, individuato i punti da migliorare e definito un piano di intervento. Sulla dashboard (front end e back end) sono stati eseguiti typecheck, test automatici e build; il server è stato avviato e provato end to end.

	Project Hub	Customer Journey Dashboard
Tipo	App mobile (iOS e Android)	Applicazione web full stack
Tecnologie	React Native 0.88, Expo SDK 58, Expo Router, TypeScript, AsyncStorage	React 18, TypeScript, Vite, Recharts, Node.js, Express, SQLite
Cosa dimostra	Interfacce mobile, state e dati derivati, persistenza locale	Modello dati, query SQL di analisi, API, test, grafici e KPI
Dati	Inseriti dall'utente e salvati sul dispositivo	Sintetici, generati in modo deterministico

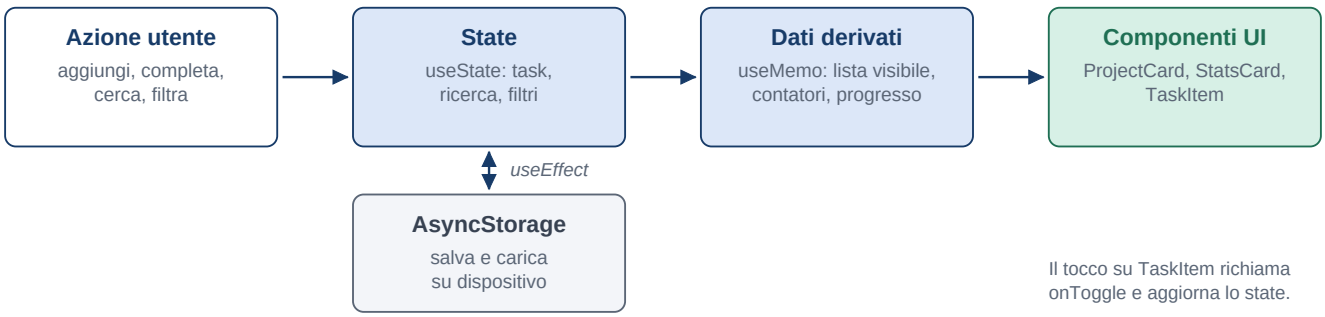
1. Project Hub: app mobile con React Native

Obiettivo. Tenere sotto controllo le attività di un progetto: avanzamento complessivo, stato di ogni task, ricerca e filtri, con i dati che restano salvati alla chiusura dell'app.

Funzionalità

- Elenco dei task con categoria; aggiunta con controllo dei campi obbligatori; completamento con un tocco.
- Barra di avanzamento e tre contatori (totali, completati, rimanenti), sempre allineati ai dati.
- Ricerca per nome o categoria (insensibile a maiuscole e accenti), filtri Tutti / Completati / Rimanenti, ordinamento A-Z e Z-A.
- Salvataggio persistente con AsyncStorage e caricamento all'avvio.

Come funziona



Concetti applicati

- **State con hook** e aggiornamenti funzionali, per evitare dati obsoleti.
- **Dati derivati**: contatori, percentuale e lista filtrata sono calcolati dallo state e non salvati, così non possono andare fuori sincrono. La lista usa `useMemo`.

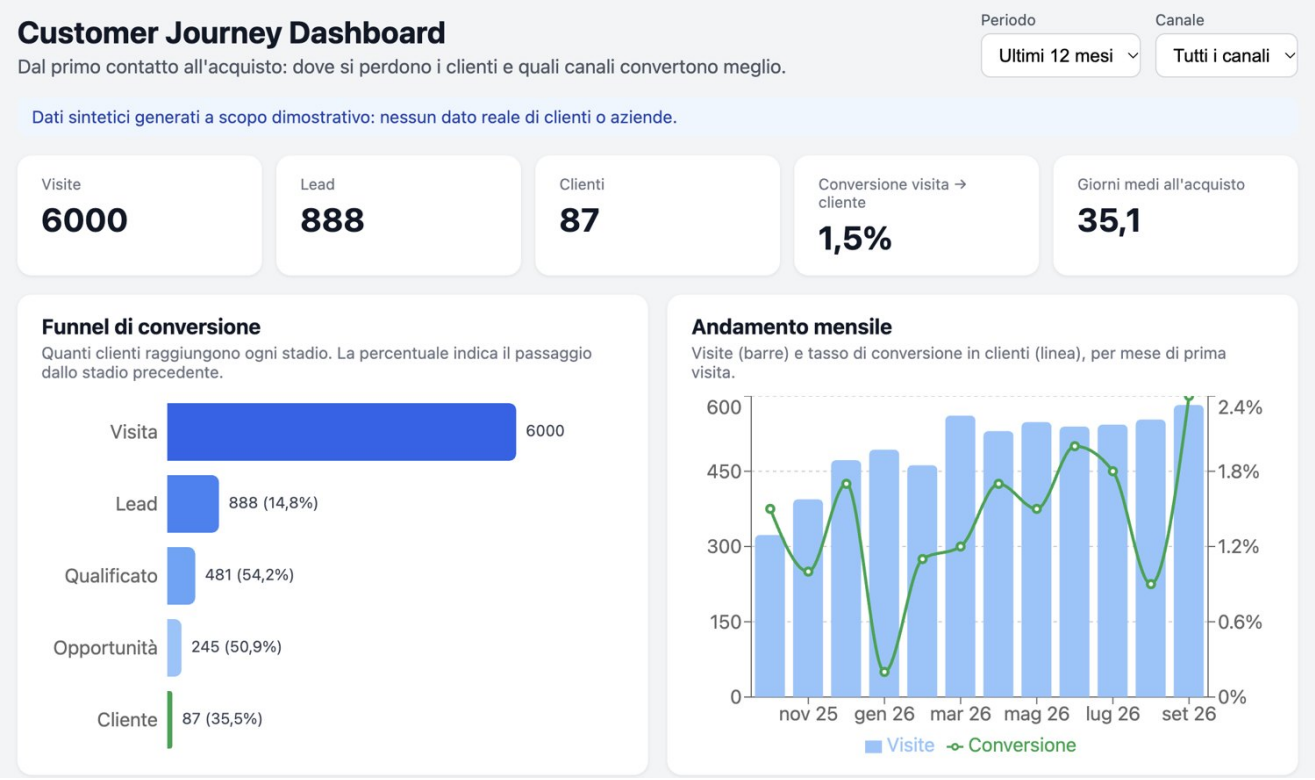
- **Persistenza sicura**: i dati letti dal dispositivo vengono validati, e il salvataggio parte solo dopo il caricamento iniziale, per non sovrascrivere quelli esistenti.
- **Componenti presentazionali tipizzati** (ProjectCard, StatsCard, TaskItem) e accessibilità di base (ruoli e stati).

Revisione e piano di miglioramento

Area	Osservazione	Intervento previsto
Layout	Margine superiore fisso, non adatto ai dispositivi con notch	SafeAreaView (react-native-safe-area-context)
Tema	Colori solo chiari ma tema di sistema automatico	Fissare il tema chiaro o supportare il dark mode con una palette centralizzata
Funzionalità	Mancano modifica ed eliminazione dei task	Eliminazione con conferma e schermata di dettaglio con rotta dinamica (Expo Router)
Struttura	Una sola schermata concentra molta logica	Estrarre un hook useTasks e i componenti AddTaskForm e TaskFilters
Qualità	Nessun test automatico	Test con Jest e React Native Testing Library

2. Customer Journey Dashboard: applicazione web con React

Obiettivo. Rispondere a due domande tipiche dell'analisi commerciale: *dove si perdono i clienti* lungo il funnel (visita, lead, qualificato, opportunità, cliente) e *quali canali convertono meglio*.



L'applicazione in esecuzione in locale (dati sintetici): KPI, funnel e andamento mensile.

Architettura



Analisi per coorte: si considerano i clienti la cui prima visita cade nel periodo scelto
e se ne seguono gli eventi, così le percentuali sono confrontabili tra periodi e canali.

Confronto tra canali

Ordinati per conversione da visita a cliente. Il filtro per canale non si applica a questa tabella.

Canale	Visite	Lead	Clienti	Visita → lead	Visita → cliente
Referral	472	107	18	22,7%	3,8%
Email	909	179	25	19,7%	2,8%
Organico	1889	306	30	16,2%	1,6%
Advertising	1285	143	8	11,1%	0,6%
Social	1445	153	6	10,6%	0,4%

Confronto tra canali: nel periodo mostrato Referral ed Email convertono meglio, Social e Advertising peggio.

Scelte progettuali

- **Analisi per coorte**: numeratore e denominatore riguardano le stesse persone, quindi i tassi sono confrontabili.
- **Aggregazioni nel database** con query a parametri nominati: al server arrivano poche righe riassuntive e nessun valore dell'utente entra nel testo SQL.
- **API in sola lettura** con validazione dei parametri (errori 400 chiari) e gestione centralizzata degli errori.
- **Hook useAsync** che annulla la richiesta precedente quando cambiano i filtri, così una risposta lenta non ne sovrascrive una più recente.
- Gestione esplicita di caricamento, errore, periodo senza dati e intervallo di date non valido; esportazione CSV compatibile con Excel in italiano.

Verifiche eseguite sulla dashboard (4 novembre 2026)

Controllo	Front end	Back end
Typecheck e build	Nessun errore, build di produzione completata	Nessun errore, compilazione completata
Test automatici	14 su 14 superati (Vitest, Testing Library)	18 su 18 superati (query SQL e API REST)
Vulnerabilità nelle dipendenze di produzione (npm audit)	0 rilevate	0 rilevate
Prova end to end	Pagina servita dal server compilato (HTTP 200)	Funnel, canali e andamento restituiti dalle API identici a quelli mostrati (6000 visite, 888 lead, 87 clienti); database integro
Coerenza dei dati	La somma per canale coincide con i totali del funnel	Percentuali ricalcolate a mano corrispondenti

Nota sul metodo. Come indicato nel README del progetto, il codice iniziale, i test e la documentazione sono stati realizzati con l'assistenza di strumenti di intelligenza artificiale; studio, esecuzione e ampliamento sono a mio carico.

3. Cosa ho imparato e prossimi passi

I due progetti condividono il modello di React (componenti, props, state, hook, TypeScript) ma cambiano gli strumenti di superficie:

Aspetto	React (web)	React Native (mobile)
Elementi di interfaccia	div, p, button, table	View, Text, Pressable, ScrollView
Stili	File CSS con classi	StyleSheet: oggetti JavaScript, layout con flexbox
Navigazione	Pagina singola	Expo Router, basato sui file della cartella app
Persistenza	Database dietro un'API REST	AsyncStorage sul dispositivo
Esecuzione	Browser (Vite)	Expo su simulatore o dispositivo

Limiti noti

- Dashboard: dati sintetici, SQLite adatto a una demo, nessuna autenticazione, funnel che assume stadi raggiunti in ordine.
- Project Hub: nessun test automatico e schermata unica da scomporre (vedi piano sopra).

Prossimi passi

- Applicare il piano di miglioramento di Project Hub e aggiungere i test.
- Dashboard: confronto tra due periodi e importazione di dati da CSV.
- Estendere i controlli automatici (test end-to-end) e valutare PostgreSQL con autenticazione per un uso reale.